

Machine Learning & Algorithmic Deep Dive

Institutional Fraud Detection Engine — Model Architecture & Mathematics

SECTION 1: ALGORITHMIC FOUNDATION & MODEL SELECTION

In institutional finance, tabular transaction data is notoriously imbalanced (fraudulent transactions typically account for < 1% of global volume) and features complex non-linear interactions between attributes (e.g., a \$100 transaction 500 miles away might be normal, but paired with a velocity of 8 transactions per hour, it becomes high risk).

Why XGBoost (eXtreme Gradient Boosting)?

- **Tree-Based Splitting on Tabular Data:** Gradient Boosted Decision Trees (GBDTs) consistently outperform deep neural networks on tabular datasets with heterogeneous numeric and categorical features.
- **Sparsity-Aware Split Finding:** XGBoost has a built-in default direction algorithm that handles missing or default feature values automatically without destructive imputation.
- **Built-In Regularization:** L1 (α) and L2 (λ) penalty terms control tree complexity and prevent overfitting on noisy financial data.

SECTION 2: MATHEMATICAL FORMULATION & OBJECTIVE FUNCTION

At its core, XGBoost minimizes a regularized objective function at step t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Where ℓ is the Binary Cross-Entropy loss, and $\Omega(f_t)$ is the complexity regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (\text{where } T \text{ is leaves, } w \text{ is leaf weight})$$

SECTION 3: TAYLOR APPROXIMATION & PRODUCTION ALIGNMENT

Second-Order Taylor Approximation

To optimize arbitrary differentiable loss functions efficiently, XGBoost uses a second-order Taylor expansion:

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}^{(t-1)}) + g_{i,t}(x_i) + \frac{1}{2} h_{i,t}^2(x_i) \right] + \Omega(f_t)$$

Where g_i and h_i represent the first and second-order gradient statistics (residuals and Hessians), allowing exact split scoring without exhaustive greedy searches.

Production Feature Alignment (Preventing Training-Serving Skew)

To prevent dimensional mismatch between API requests and trained model artifacts, incoming categorical features are encoded and strictly reindexed against expected feature matrices:

```
Code Logic: encoded_df = pd.get_dummies(input_df)
aligned_df = encoded_df.reindex(columns=model_feature_names, fill_value=0)
```

SECTION 4: TECHNICAL INTERVIEW Q&A

Q: How do you handle severe class imbalance in fraud datasets?

Answer: "Fraud is heavily imbalanced (< 1%). We handle this during training by tuning `scale_pos_weight` in XGBoost to balance class weights, optimizing for **Precision-Recall AUC (PR-AUC)** rather than raw accuracy, and establishing calibrated decision thresholds."

Q: Why serialize models with joblib instead of retraining on the fly?

Answer: "Inference latency is critical—financial gateways require sub-200ms responses. Retraining on the fly is computationally prohibitive and non-deterministic. Serializing boosters and loading them once via FastAPI's lifespan context guarantees zero-disk-I/O high-performance inference."